

Practical Work 5: The Longest Path with MapReduce

Le Quy Ton
23BI14425

December 15, 2025

1 Introduction

This practical work focuses on finding the longest file path(s) within a distributed file system simulation using the MapReduce paradigm. The goal is to process a set of file paths (simulating output from `find /`) and efficiently identify the path string with the maximum character count.

2 Implementation Choice

Similar to the previous Word Count exercise, we utilize the *Python Streaming* approach combined with Linux utilities, adhering to the "Invent yourself" requirement.

- **Mapper:** Written in Python. It calculates the length of each path string.
- **Shuffle/Sort:** Utilizes the Linux `sort -n -r` command. The flags `-n` (numeric) and `-r` (reverse) are crucial to place the longest paths at the top of the stream.
- **Reducer:** Written in Python. It reads the sorted stream and outputs the top element(s).

3 System Design and Operation

3.1 Mapper Logic

The Mapper reads file paths line by line. For each path, it computes the string length and emits a key-value pair: `< Length, Path >`. Example: Input `/usr/bin`

→ Output 8\t/usr/bin.

3.2 Reducer Logic

The Reducer receives input sorted by Length in descending order. It captures the length of the very first record (the maximum length). It then prints every path matching this length and terminates immediately upon encountering a shorter path, ensuring efficiency.

3.3 Process Visualization

Figure 1 illustrates the data flow for finding the longest path.

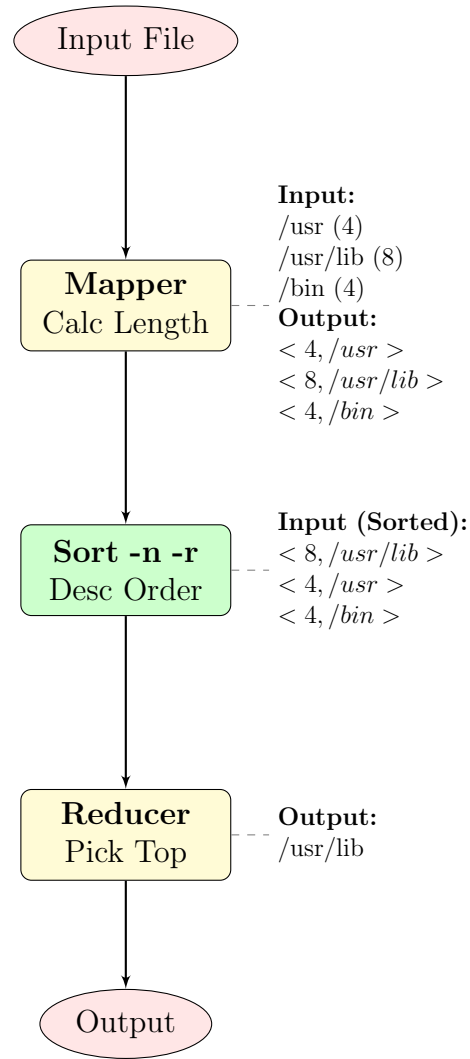


Figure 1: Workflow of the Longest Path MapReduce

4 Execution and Results

We verified the implementation by creating a sample input file containing paths of various lengths. The pipeline was executed using the standard Linux command line. Figure 2 shows the terminal output. The system correctly identified `/home/user/documents/report.pdf` as the longest path from the provided input list.

```

tonle@cust-debian:~/LongestPath$ cat <<EOF > input.txt
/usr/bin
/usr/local/bin/python3
/home/user/documents/report.pdf
/var/log/syslog
/tmp/short
EOF
tonle@cust-debian:~/LongestPath$ chmod +x mapper.py reducer.py
tonle@cust-debian:~/LongestPath$ cat input.txt | ./mapper.py | sort -n -r | ./reducer.py
/home/user/documents/report.pdf
tonle@cust-debian:~/LongestPath$ cat ./input.txt
/usr/bin
/usr/local/bin/python3
/home/user/documents/report.pdf
/var/log/syslog
/tmp/short

```

Figure 2: Successful execution of Longest Path MapReduce

5 Code Implementation

5.1 Mapper (mapper.py)

```

1 import sys
2 for line in sys.stdin:
3     path = line.strip()
4     if path:
5         # Emit length first for sorting
6         print(f"{len(path)}\t{path}")

```

5.2 Reducer (reducer.py)

```

1 import sys
2 max_len = -1
3 for line in sys.stdin:
4     try:
5         len_str, path = line.strip().split('\t', 1)
6         length = int(len_str)
7
8         if max_len == -1: max_len = length
9
10        if length == max_len:
11            print(path) # Found a longest path
12        elif length < max_len:
13            break # Stop processing since list is sorted desc

```

```
14 | except ValueError: continue
```

6 Conclusion

We successfully implemented the Longest Path problem using a MapReduce pipeline. The key difference from Word Count is the sorting strategy: by emitting the length as the key and sorting numerically in reverse (`sort -n -r`), the Reducer only needs to process the first few lines to find the global maximum. This demonstrates the flexibility of the MapReduce model for different types of aggregation tasks.